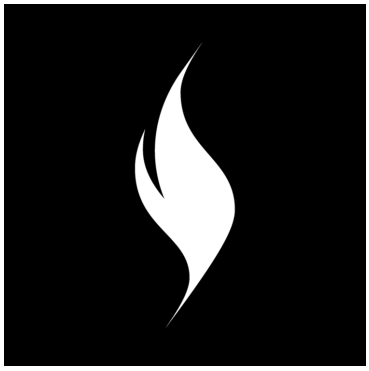

CosmWasm Smart Contract Audit Report

Security Assessment

CONTRACT

user_map

AUDIT DATE

2026-06-15

VERSION

1.0

Prepared by

CosmWasm AI Auditor by Burnt Labs

1. Introduction

1.1 Scope

A simple single-contract CosmWasm workspace that acts as a decentralized key-value map keyed by user address. Any user can store a JSON-validated string value under their own address and anyone can query individual values, paginated lists of users, or the full map. The audit covered one contract in the `user_map` workspace located at `../contracts/contracts/user_map`

1.2 Submitted Codebase

Item	Value
Repository	../contracts/contracts/user_map
Commit	7315e195d6c8c17347e5a0dd999ce4858310a6cd

2. Code Criteria

Criterion	Status	Notes
Documentation	Satisfactory	Entry points and message types are documented; some functions lack inline documentation.
Coverage	Satisfactory	Basic test coverage exists for core functionality; edge cases around migration could be expanded.
Readability	Good	Clear naming conventions and modular contract structure make the codebase easy to follow.
Complexity	Good	The contract is simple and well-factored with minimal nested logic.

3. Findings Summary

Total Findings	1
Critical	0
Severe	0
Moderate	0
Low	1
Informational	0

Findings Table

#	Title	Severity	Status
1	migrate() has no contract name or version progression check	LOW	confirmed

4. Findings Technical Details

Finding 1: migrate() has no contract name or version progression check

Severity: LOW (Likelihood: 2/5 × Impact: 4/5) | **Contract:** user-map | **Location:** src/contract.rs:26-29 | **Status:** confirmed

Description

The `migrate` entry point performs no validation of the existing contract identity or version. It does not call `cw2::get_contract_version` to verify the stored contract name matches `CONTRACT_NAME`, nor does it compare versions to prevent downgrades. While `migrate` is admin-gated at the SDK level, the absence of an in-contract version guard violates defense-in-depth best practices.

Impact

A compromised or negligent admin could downgrade the contract to an older vulnerable version or `migrate` from an unrelated contract type, causing state corruption or re-introducing patched vulnerabilities with no on-chain guardrail.

Recommendation

In `migrate()`, call `cw2::get_contract_version` to verify the stored contract name matches `CONTRACT_NAME` and enforce that the target version is strictly newer than the currently stored version using semver comparison.

Code Snippet

```
#[entry_point]
pub fn migrate(_deps: DepsMut, _env: Env, _msg: MigrateMsg) ->
ContractResult<Response> {
    Ok(Response::default())
}
```

5. Good Practices

- Contract uses `cw2` for contract version tracking at instantiation
- Admin authorization for `migrate` is properly handled at the SDK level
- JSON validation is enforced on stored values, preventing malformed data
- Clean and modular contract structure with clear separation of concerns

6. Recommendations

- Add version progression checks in `migrate()` to prevent contract downgrades and mismatched migrations
- Expand test coverage to include migration scenarios and edge cases
- Consider adding comprehensive integration tests for all query entry points with large datasets

7. Document Control

Version	Date	Author	Changes
1.0	2026-06-15	CosmWasm AI Auditor by Burnt Labs	Initial Report

APPENDIX —

Appendix A: Risk Assessment Methodology

Severity is determined using a Likelihood × Impact matrix:

- **Likelihood (1-5):** How easily can the vulnerability be exploited?
- **Impact (1-5):** What is the potential damage if exploited?

Severity mapping:

- **CRITICAL (10):** Trivial exploit, catastrophic impact
- **SEVERE (8-9):** Feasible exploit, high impact
- **MODERATE (6-7):** Possible exploit, moderate impact
- **LOW (4-5):** Difficult exploit, low impact
- INFORMATIONAL (1-3): Best practice recommendation

APPENDIX —

Appendix B: Disclaimer

This audit report is provided for informational purposes only and does not constitute a guarantee that the code is free from vulnerabilities. The audit was conducted using AI-powered analysis tools and manual review techniques. No warranty is made regarding the completeness or accuracy of this report.

APPENDIX —

Appendix C: Static Analysis Results

Static analysis was skipped or unavailable.

APPENDIX —

Appendix D: LLM Pipeline

D.1 Methodology

AI-powered analysis using CrewAI + LiteLLM. The audit approach included static analysis, AI-driven vulnerability detection across all entry points, and best-practice checklist coverage. Reasoning-based verification evaluates boundary conditions and attack scenarios but does not execute exploit code. Off-chain components, frontend code, and deployment infrastructure are out of scope. Economic modeling and game theory analysis are not included.

D.2 Model Usage

This audit used multiple LLM models for redundancy. The table below records which model was used for each pipeline phase.

Phase	Model Used	Switched	Models Tried
audit:user-map	`openai/glm-5.2`	No	-
hunt:A:user-map	`openai/glm-5.2`	No	-
hunt:B:user-map	`openai/glm-5.2`	No	-
hunt:C:user-map	`openai/glm-5.2`	No	-
hunt:D:user-map	`openai/glm-5.2`	No	-
hunt:E:user-map	`openai/glm-5.2`	No	-
hunt:F:user-map	`openai/glm-5.2`	No	-
recon	`openai/glm-5.2`	No	-
report	`openai/glm-5.2`	No	-
triage	`openai/glm-5.2`	No	-

D.3 Output Reliability

- Overall score: 100/100

- Degraded phases: 0

Report generated by CosmWasm AI Auditor